ShopKart E-Commerce Business Analytics

From Database Design to Strategic Business Intelligence

Submitted by: Chetan Prajapat

File: ACE_Project_ChetanPrajapat

Course: BCA 4th Sem BCA
College: Sage university indore
Enrolment no: 24COA2BCA0044

GitHub: https://github.com/grchetan/ACE_Project_Query_ChetanPrajapat/

1. Introduction — Meet ShopKart

ShopKart is an online marketplace operating across major Indian cities, selling products across three core categories. This case study demonstrates a complete SQL Data Analyst workflow — from database design and data insertion to multi-level business intelligence queries.

1.1 Product Categories

Category	Products	Price Range
Electronics	Laptops, Mobiles, Headphones	High-value tech products
Fashion	T-Shirts, Jeans	High volume, lower price-point
Home Appliances	Microwave, Refrigerator, Air Conditioner	Large-ticket, strong margins

1.2 The Management Problem

Despite growing order volumes, ShopKart's leadership faces five critical gaps that require data-driven answers:

- Profit Clarity — Sales are increasing, but profitability remains unclear. Management cannot distinguish high-revenue products from high-profit ones.
- City-Level Anomalies — Certain cities show high order counts but surprisingly low revenue.
- Top Customer Identification — The business needs to identify its most valuable customers for targeted retention strategies.
- Category Performance — No clear view of which product categories are driving profit versus just volume.
- Monthly Growth Tracking — A structured monthly order trend report is needed to understand growth trajectory.

2. Step 1 — Create the Database

The first step is to initialize the ShopKart database environment in MySQL. All subsequent tables and data will live within this database.

```
CREATE DATABASE shopkart_db;
USE shopkart_db;
```

This establishes the working schema. The USE statement ensures all subsequent DDL and DML commands are executed within the correct database context — a critical step before any table creation or data insertion.

3. Step 2 — Database Schema Design

ShopKart's relational database is built on five interconnected tables. Each table has a clearly defined primary key, and foreign keys enforce referential integrity across the schema.

3.1 Schema Overview

Table	Primary Key	Foreign Key(s)	Purpose
customers	customer_id	—	Customer profiles and demographics
categories	category_id	—	Product category hierarchy
products	product_id	category_id → categories	Product catalog with price and cost
orders	order_id	customer_id → customers	Order header with status and city
order_items	order_item_id	order_id → orders, product_id → products	Line items linking orders and products

The schema follows a classic star-like structure: Orders sits at the center, linked to Customers (who placed the order), Order Items (what was ordered), and Products (which belong to Categories). This design enables powerful multi-table JOIN queries for business analysis.

3.2 Table Creation — DDL

Customers Table

```
CREATE TABLE customers (
    customer_id   INT PRIMARY KEY,
    customer_name VARCHAR(100),
    gender        VARCHAR(10),
    city          VARCHAR(50),
    signup_date   DATE
);
```

Categories Table

```
CREATE TABLE categories (
    category_id   INT PRIMARY KEY,
    category_name VARCHAR(50)
);
```

Products Table

```
CREATE TABLE products (
    product_id   INT PRIMARY KEY,
    product_name VARCHAR(100),
    category_id  INT,
    price        DECIMAL(10,2),
    cost         DECIMAL(10,2),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

Orders Table

```
CREATE TABLE orders (
    order_id     INT PRIMARY KEY,
    customer_id  INT,
    order_date   DATE,
    city         VARCHAR(50),
    order_status VARCHAR(20),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Order Items Table

```
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id      INT,
    product_id    INT,
    quantity      INT,
    FOREIGN KEY (order_id)    REFERENCES orders(order_id),
    FOREIGN KEY (product_id)  REFERENCES products(product_id)
);
```

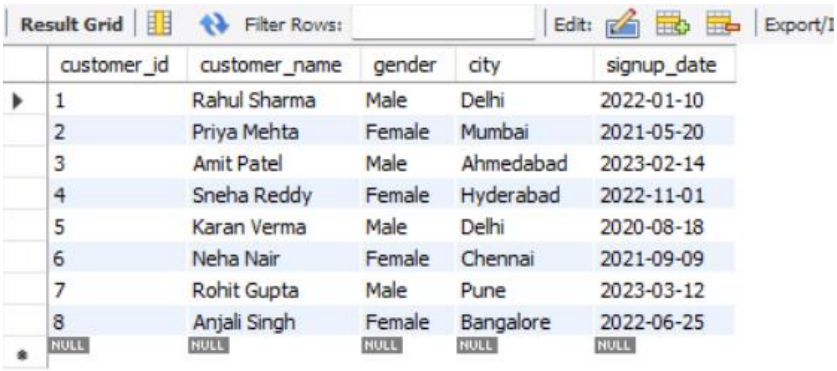
4. Step 3 — Insert Dataset

The following INSERT statements populate all five tables with ShopKart's sample dataset.

4.1 Customers Data

```
INSERT INTO customers VALUES
  (1, 'Rahul Sharma',  'Male',   'Delhi',     '2022-01-10'),
  (2, 'Priya Mehta',   'Female', 'Mumbai',    '2021-05-20'),
  (3, 'Amit Patel',    'Male',   'Ahmedabad', '2023-02-14'),
  (4, 'Sneha Reddy',   'Female', 'Hyderabad', '2022-11-01'),
  (5, 'Karan Verma',   'Male',   'Delhi',     '2020-08-18'),
  (6, 'Neha Nair',     'Female', 'Chennai',   '2021-09-09'),
  (7, 'Rohit Gupta',   'Male',   'Pune',      '2023-03-12'),
  (8, 'Anjali Singh',  'Female', 'Bangalore', '2022-06-25');
```

Output:

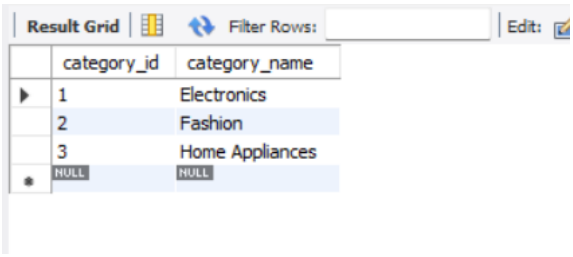


customer_id	customer_name	gender	city	signup_date
1	Rahul Sharma	Male	Delhi	2022-01-10
2	Priya Mehta	Female	Mumbai	2021-05-20
3	Amit Patel	Male	Ahmedabad	2023-02-14
4	Sneha Reddy	Female	Hyderabad	2022-11-01
5	Karan Verma	Male	Delhi	2020-08-18
6	Neha Nair	Female	Chennai	2021-09-09
7	Rohit Gupta	Male	Pune	2023-03-12
8	Anjali Singh	Female	Bangalore	2022-06-25
NULL	NULL	NULL	NULL	NULL

4.2 Categories Data

```
INSERT INTO categories VALUES
  (1, 'Electronics'),
  (2, 'Fashion'),
  (3, 'Home Appliances');
```

Output:

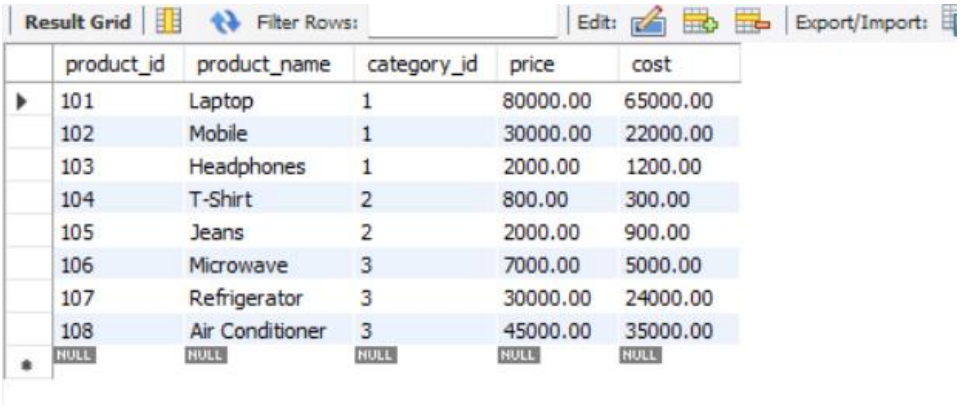


category_id	category_name
1	Electronics
2	Fashion
3	Home Appliances
NULL	NULL

4.3 Products Data

```
INSERT INTO products VALUES
  (101, 'Laptop',           1, 80000, 65000),
  (102, 'Mobile',           1, 30000, 22000),
  (103, 'Headphones',       1,  2000,  1200),
  (104, 'T-Shirt',          2,   800,   300),
  (105, 'Jeans',            2,  2000,   900),
  (106, 'Microwave',        3,  7000,  5000),
  (107, 'Refrigerator',     3, 30000, 24000),
  (108, 'Air Conditioner',  3, 45000, 35000);
```

Output:



product_id	product_name	category_id	price	cost
101	Laptop	1	80000.00	65000.00
102	Mobile	1	30000.00	22000.00
103	Headphones	1	2000.00	1200.00
104	T-Shirt	2	800.00	300.00
105	Jeans	2	2000.00	900.00
106	Microwave	3	7000.00	5000.00
107	Refrigerator	3	30000.00	24000.00
108	Air Conditioner	3	45000.00	35000.00
NULL	NULL	NULL	NULL	NULL

4.4 Orders Data

```
INSERT INTO orders VALUES
  (1001, 1, '2024-01-10', 'Delhi',     'Delivered'),
  (1002, 2, '2024-01-11', 'Mumbai',    'Delivered'),
  (1003, 3, '2024-01-12', 'Ahmedabad', 'Cancelled'),
  (1004, 4, '2024-01-13', 'Hyderabad', 'Delivered'),
  (1005, 5, '2024-01-15', 'Delhi',     'Delivered'),
  (1006, 6, '2024-01-16', 'Chennai',   'Pending'),
  (1007, 1, '2024-02-02', 'Delhi',     'Delivered'),
  (1008, 7, '2024-02-05', 'Pune',      'Delivered'),
  (1009, 8, '2024-02-08', 'Bangalore', 'Delivered'),
  (1010, 2, '2024-02-10', 'Mumbai',    'Cancelled');
```

Output:

Result Grid | Filter Rows: | Edit: | Export/Imp

order_id	customer_id	order_date	city	order_status
1001	1	2024-01-10	Delhi	Delivered
1002	2	2024-01-11	Mumbai	Delivered
1003	3	2024-01-12	Ahmedabad	Cancelled
1004	4	2024-01-13	Hyderabad	Delivered
1005	5	2024-01-15	Delhi	Delivered
1006	6	2024-01-16	Chennai	Pending
1007	1	2024-02-02	Delhi	Delivered
1008	7	2024-02-05	Pune	Delivered
1009	8	2024-02-08	Bangalore	Delivered
1010	2	2024-02-10	Mumbai	Cancelled
NULL	NULL	NULL	NULL	NULL

4.5 Order Items Data

```
INSERT INTO order_items VALUES
  (1,  1001, 101, 1),    -- Order 1001: Laptop x1
  (2,  1001, 103, 2),    -- Order 1001: Headphones x2
  (3,  1002, 102, 1),    -- Order 1002: Mobile x1
  (4,  1003, 104, 3),    -- Order 1003: T-Shirt x3 (Cancelled)
  (5,  1004, 106, 1),    -- Order 1004: Microwave x1
  (6,  1005, 105, 2),    -- Order 1005: Jeans x2
  (7,  1005, 104, 1),    -- Order 1005: T-Shirt x1
  (8,  1006, 102, 1),    -- Order 1006: Mobile x1 (Pending)
  (9,  1007, 108, 1),    -- Order 1007: Air Conditioner x1
  (10, 1008, 103, 3),    -- Order 1008: Headphones x3
  (11, 1009, 107, 1),    -- Order 1009: Refrigerator x1
  (12, 1010, 104, 2);    -- Order 1010: T-Shirt x2 (Cancelled)
```

Output:

order_item_id	order_id	product_id	quantity
1	1001	101	1
2	1001	103	2
3	1002	102	1
4	1003	104	3
5	1004	106	1
6	1005	105	2
7	1005	104	1
8	1006	102	1
9	1007	108	1
10	1008	103	3
11	1009	107	1
12	1010	104	2
NULL	NULL	NULL	NULL

order_items 6 ×

5. Level 1 — Basic Exploration Queries

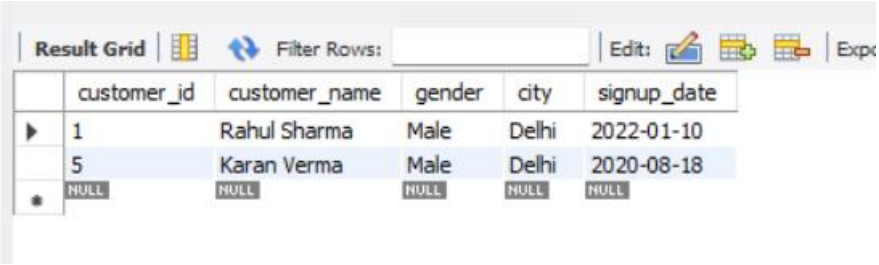
Level 1 establishes foundational SQL skills — filtering, sorting, and limiting results. These queries form the bedrock of any data investigation.

Q1 — Customers from Delhi

Returns all customers whose city is Delhi. Uses the WHERE clause to filter rows by city value.

```
SELECT *
FROM customers
WHERE city = 'Delhi';
```

Result:



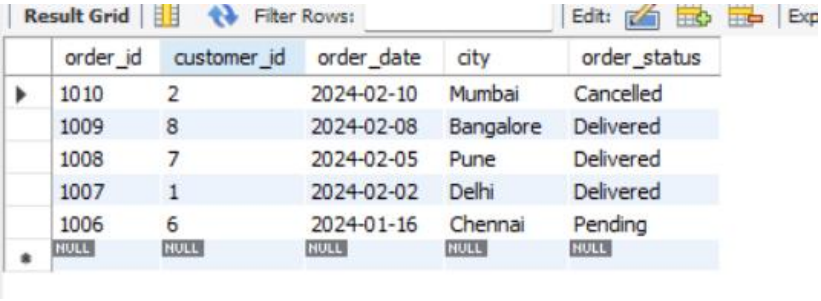
customer_id	customer_name	gender	city	signup_date
1	Rahul Sharma	Male	Delhi	2022-01-10
5	Karan Verma	Male	Delhi	2020-08-18
NULL	NULL	NULL	NULL	NULL

Q2 — Latest 5 Orders

Sorts all orders by order_date in descending order and returns only the 5 most recent. Useful for operational dashboards.

```
SELECT *
FROM orders
ORDER BY order_date DESC
LIMIT 5;
```

Result:



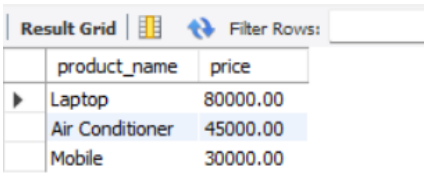
order_id	customer_id	order_date	city	order_status
1010	2	2024-02-10	Mumbai	Cancelled
1009	8	2024-02-08	Bangalore	Delivered
1008	7	2024-02-05	Pune	Delivered
1007	1	2024-02-02	Delhi	Delivered
1006	6	2024-01-16	Chennai	Pending
NULL	NULL	NULL	NULL	NULL

Q3 — Top 3 Most Expensive Products

Returns the three highest-priced items in the catalog using ORDER BY price DESC and LIMIT 3.

```
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 3;
```

Result:



product_name	price
Laptop	80000.00
Air Conditioner	45000.00
Mobile	30000.00

6. Level 2 — Aggregation Queries

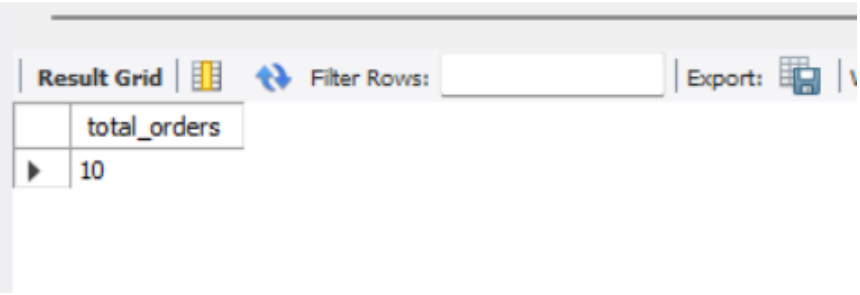
Level 2 introduces aggregate functions and grouping — the core of business reporting. These queries answer 'how many' and 'how much' questions at scale.

Q4 — Total Number of Orders

Uses COUNT() to return the total order volume. This is the simplest aggregation and a baseline KPI for any e-commerce business.

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Result:

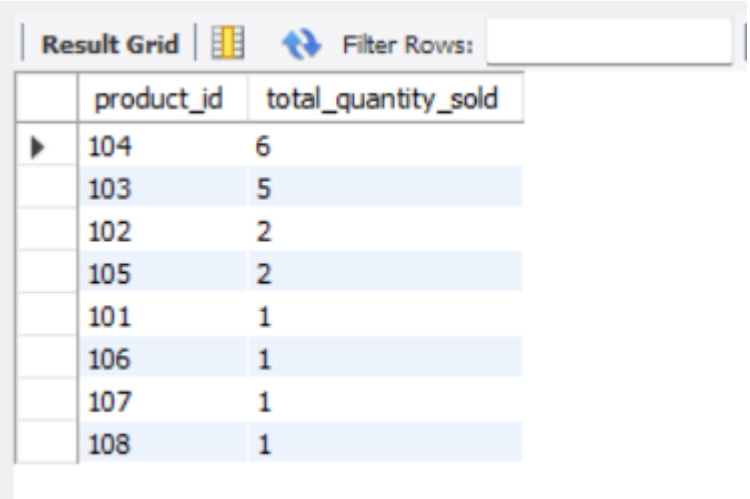


Q5 — Total Quantity Sold Per Product

Groups order line items by product and sums quantities. Reveals which products move the most units — critical for inventory planning.

```
SELECT product_id,
SUM(quantity) AS total_quantity_sold
FROM order_items
GROUP BY product_id
ORDER BY total_quantity_sold DESC;
```

Result:



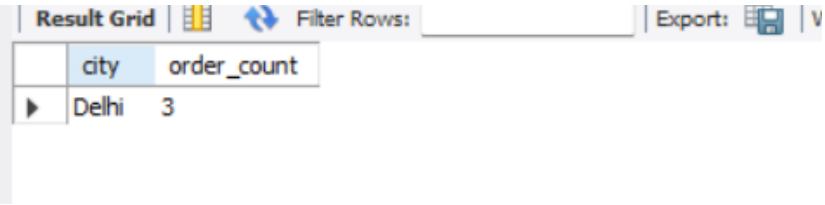
Note: product_id 102 (Mobile) appears in both a delivered (1002) and a pending order (1006).

Q6 — Cities with More Than 2 Orders

Uses HAVING (not WHERE) to filter after aggregation. HAVING is used when filtering on aggregate functions like COUNT().

```
SELECT city, COUNT(*) AS order_count
FROM orders
GROUP BY city
HAVING COUNT(*) > 2;
```

Result:



7. Level 3 — Join-Based Real Analysis

Level 3 unlocks the true power of relational databases. By joining multiple tables, we can answer complex business questions that no single table can answer alone.

Q7 — Customer Name with Order ID

Joins customers and orders tables on the shared customer_id key. Produces a human-readable order list with customer names instead of raw IDs.

```
SELECT c.customer_name, o.order_id, o.order_date, o.city, o.order_status
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
ORDER BY o.order_date;
```

Result:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id	order_date	city	order_status
Rahul Sharma	1001	2024-01-10	Delhi	Delivered
Priya Mehta	1002	2024-01-11	Mumbai	Delivered
Amit Patel	1003	2024-01-12	Ahmedabad	Cancelled
Sneha Reddy	1004	2024-01-13	Hyderabad	Delivered
Karan Verma	1005	2024-01-15	Delhi	Delivered
Neha Nair	1006	2024-01-16	Chennai	Pending
Rahul Sharma	1007	2024-02-02	Delhi	Delivered
Rohit Gupta	1008	2024-02-05	Pune	Delivered
Anjali Singh	1009	2024-02-08	Bangalore	Delivered
Priya Mehta	1010	2024-02-10	Mumbai	Cancelled

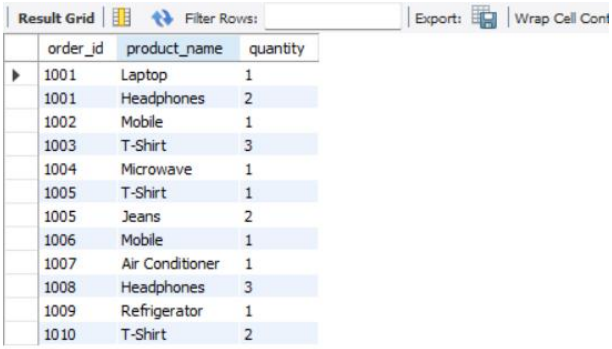
Q8 — Product Name in Each Order

A three-table JOIN spanning orders, order_items, and products. This is the most common pattern in e-commerce analytics — the order receipt view.

```
SELECT o.order_id, p.product_name, oi.quantity
```

```
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
ORDER BY o.order_id;
```

Result:



Result Grid | Filter Rows: | Export: | Wrap Cell Cont

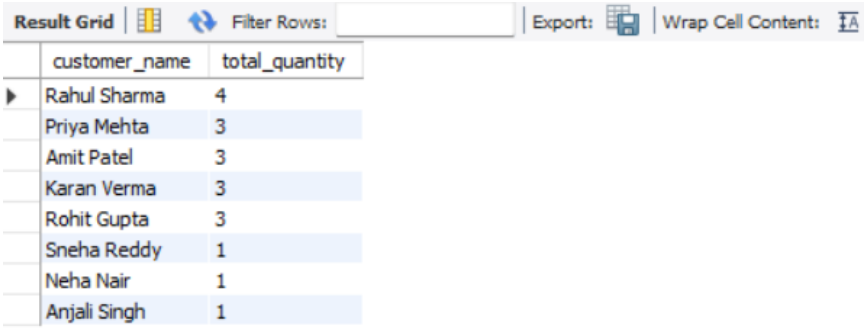
order_id	product_name	quantity
1001	Laptop	1
1001	Headphones	2
1002	Mobile	1
1003	T-Shirt	3
1004	Microwave	1
1005	T-Shirt	1
1005	Jeans	2
1006	Mobile	1
1007	Air Conditioner	1
1008	Headphones	3
1009	Refrigerator	1
1010	T-Shirt	2

Q9 — Total Quantity Per Customer

Combines JOIN with GROUP BY to aggregate total purchase quantity per customer — a foundation for customer ranking and segmentation.

```
SELECT c.customer_name, SUM(oi.quantity) AS total_quantity
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_name
ORDER BY total_quantity DESC;
```

Result:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	total_quantity
Rahul Sharma	4
Priya Mehta	3
Amit Patel	3
Karan Verma	3
Rohit Gupta	3
Sneha Reddy	1
Neha Nair	1
Anjali Singh	1

8. Level 4 — Profit Analytics

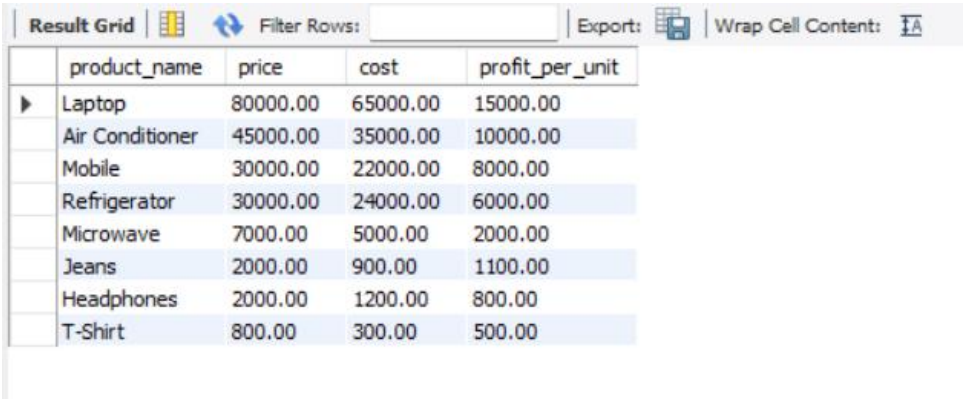
Profit analytics is one of the most industry-critical SQL skills. The key formula: Profit = (Price - Cost) x Quantity. These queries directly address management's core concern about unclear profitability.

Q10 — Profit Per Product (Per Unit)

Calculates per-unit margin for every product. This reveals that Laptop has the highest absolute margin per unit at Rs. 15,000, while T-Shirt has only Rs. 500 margin per unit.

```
SELECT product_name, price, cost,
(price - cost) AS profit_per_unit
FROM products
ORDER BY profit_per_unit DESC;
```

Result:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

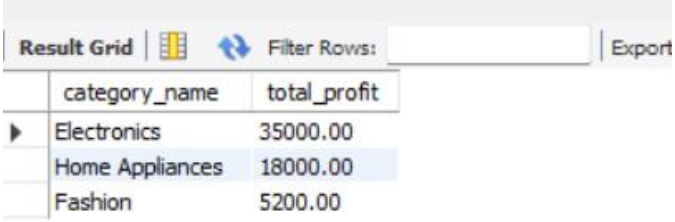
product_name	price	cost	profit_per_unit
Laptop	80000.00	65000.00	15000.00
Air Conditioner	45000.00	35000.00	10000.00
Mobile	30000.00	22000.00	8000.00
Refrigerator	30000.00	24000.00	6000.00
Microwave	7000.00	5000.00	2000.00
Jeans	2000.00	900.00	1100.00
Headphones	2000.00	1200.00	800.00
T-Shirt	800.00	300.00	500.00

Q11 — Category-Wise Total Profit

A four-table JOIN that rolls up profit to the category level. This answers the management question: which category is most profitable — Electronics, Fashion, or Home Appliances?

```
SELECT cat.category_name,
SUM((p.price - p.cost) * oi.quantity) AS total_profit
FROM categories cat
JOIN products p     ON cat.category_id = p.category_id
JOIN order_items oi ON p.product_id    = oi.product_id
JOIN orders o       ON oi.order_id     = o.order_id
GROUP BY cat.category_name
ORDER BY total_profit DESC;
```

Result:



Result Grid | Filter Rows: | Export

category_name	total_profit
Electronics	35000.00
Home Appliances	18000.00
Fashion	5200.00

9. Level 5 — Customer Intelligence

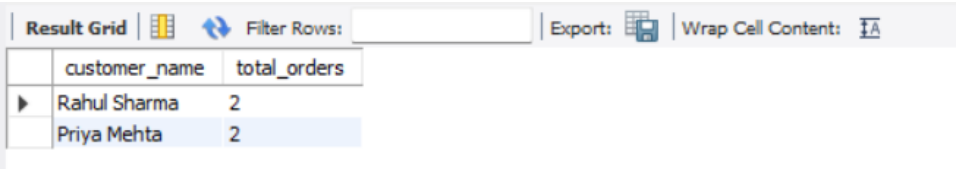
Level 5 moves from transactional reporting to customer behavioral analysis — identifying repeat buyers and high-value purchasers using advanced SQL techniques including subqueries.

Q12 — Customers with More Than 1 Order (Repeat Buyers)

Uses HAVING COUNT() after grouping to isolate repeat customers. Repeat customer rate is a key retention KPI for any e-commerce business.

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING COUNT(o.order_id) > 1;
```

Result:

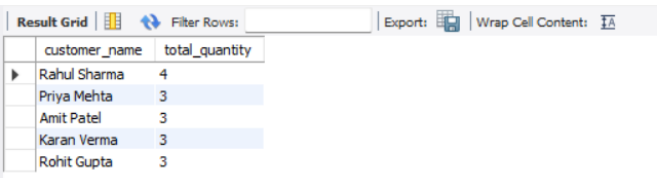

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	total_orders
Rahul Sharma	2
Priya Mehta	2

Q13 — Customers Above Average Purchase Quantity

Introduces a subquery to dynamically compute the average purchase quantity and use it as a filter threshold. This is a hallmark of intermediate-to-advanced SQL — the inner query calculates the benchmark, the outer query applies it.

```
SELECT c.customer_name, SUM(oi.quantity) AS total_quantity
FROM customers c
JOIN orders o      ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id   = oi.order_id
GROUP BY c.customer_name
HAVING SUM(oi.quantity) > (
    SELECT AVG(total_qty)
    FROM (
        SELECT SUM(quantity) AS total_qty
        FROM order_items
        GROUP BY order_id
    ) subquery
);
```


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	total_quantity
Rahul Sharma	4
Priya Mehta	3
Amit Patel	3
Karan Verma	3
Rohit Gupta	3

The inner subquery computes order-level quantity averages, and the outer query returns only customers whose total purchase volume exceeds that average — enabling dynamic threshold-based filtering.

10. Level 6 — Strategic Business Questions

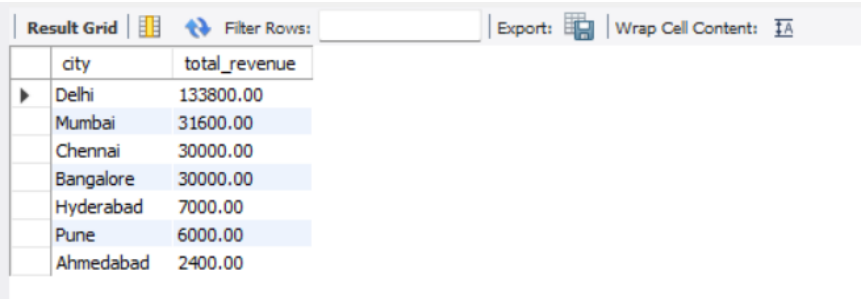
Level 6 delivers the executive-level insights that directly inform business strategy — city revenue rankings, product profitability, and growth trends over time.

Q14 — City with Highest Revenue

Addresses the management concern about cities with high orders but low revenue. Multiplies price x quantity per city and ranks them to expose geographic revenue distribution.

```
SELECT o.city,
       SUM(p.price * oi.quantity) AS total_revenue
FROM orders o
JOIN order_items oi ON o.order_id    = oi.order_id
JOIN products p     ON oi.product_id = p.product_id
GROUP BY o.city
ORDER BY total_revenue DESC;
```

Result:


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

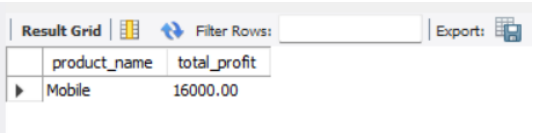
city	total_revenue
Delhi	133800.00
Mumbai	31600.00
Chennai	30000.00
Bangalore	30000.00
Hyderabad	7000.00
Pune	6000.00
Ahmedabad	2400.00

Q15 — Most Profitable Product

Combines margin calculation with quantity sold to find the product generating the most absolute profit — not just the highest price or highest volume.

```
SELECT p.product_name,
SUM((p.price - p.cost) * oi.quantity) AS total_profit
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY total_profit DESC
LIMIT 1;
```

Result:


Result Grid | Filter Rows: | Export:

product_name	total_profit
Mobile	16000.00

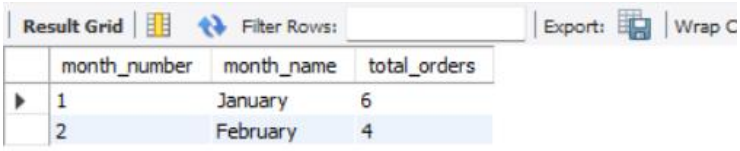
Q16 — Monthly Order Trend (Jan vs Feb 2024)

Uses MONTH() date function to extract the month from each order date and group by it. Produces the monthly growth report management requested.

```
SELECT MONTH(order_date) AS month_number,
MONTHNAME(order_date) AS month_name,
COUNT(*) AS total_orders
FROM orders
```

```
GROUP BY MONTH(order_date), MONTHNAME(order_date)
ORDER BY month_number;
```

Result:



month_number	month_name	total_orders
1	January	6
2	February	4

Business Insight: January had 6 orders and February had 4. While February shows lower volume, trend analysis requires more months of data to determine seasonality vs. decline.

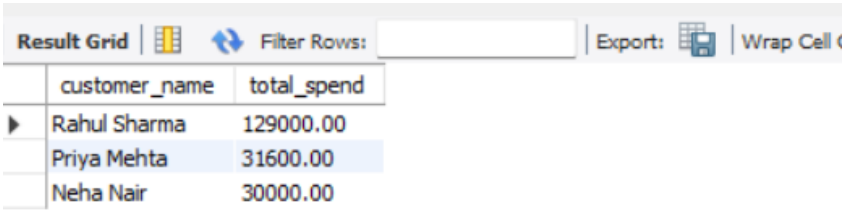
11. Final Capstone — Complete Business Intelligence Report

The manager's final requirement: a comprehensive SQL report that consolidates all analytical dimensions into a single business intelligence deliverable.

Capstone Part A — Top 3 Customers by Total Spend

```
SELECT c.customer_name,
SUM(p.price * oi.quantity) AS total_spend
FROM customers c
JOIN orders o      ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id   = oi.order_id
JOIN products p    ON oi.product_id = p.product_id
GROUP BY c.customer_name
ORDER BY total_spend DESC
LIMIT 3;
```

Expected Result:



customer_name	total_spend
Rahul Sharma	129000.00
Priya Mehta	31600.00
Neha Nair	30000.00

Capstone Part B — Revenue Per City

```
SELECT o.city,
COUNT(DISTINCT o.order_id) AS total_orders,
SUM(p.price * oi.quantity) AS total_revenue
FROM orders o
JOIN order_items oi ON o.order_id    = oi.order_id
JOIN products p     ON oi.product_id = p.product_id
GROUP BY o.city
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	total_orders	total_revenue
Delhi	3	133800.00
Mumbai	2	31600.00
Bangalore	1	30000.00
Chennai	1	30000.00
Hyderabad	1	7000.00
Pune	1	6000.00
Ahmedabad	1	2400.00

Capstone Part C — Profit Per Category (Full P&L View)

```
SELECT cat.category_name,
       SUM(p.price * oi.quantity)            AS total_revenue,
       SUM(p.cost  * oi.quantity)            AS total_cost,
       SUM((p.price - p.cost) * oi.quantity) AS total_profit,
       ROUND(
         SUM((p.price - p.cost) * oi.quantity) * 100.0 /
         SUM(p.price * oi.quantity), 2
       ) AS profit_margin_pct
FROM categories cat
JOIN products p     ON cat.category_id = p.category_id
JOIN order_items oi ON p.product_id    = oi.product_id
JOIN orders o       ON oi.order_id     = o.order_id
GROUP BY cat.category_name
ORDER BY total_profit DESC;
```

Capstone Part D — Repeat Customer Count

```
SELECT COUNT(*) AS repeat_customer_count
FROM (
    SELECT customer_id, COUNT(order_id) AS order_count
    FROM orders
    GROUP BY customer_id
    HAVING COUNT(order_id) > 1
) repeat_customers;
```

Result:

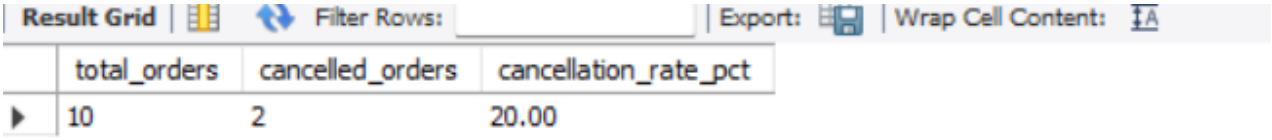
Result Grid | Filter Rows: | Export: | Wrap Cell

repeat_customer_count
2

Capstone Part E — Cancelled Order Percentage

```
SELECT
    COUNT(*) AS total_orders,
    SUM(CASE WHEN order_status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
    ROUND(
        SUM(CASE WHEN order_status = 'Cancelled' THEN 1 ELSE 0 END) * 100.0
        / COUNT(*), 2
    ) AS cancellation_rate_pct
FROM orders;
```

Result:



total_orders	cancelled_orders	cancellation_rate_pct
10	2	20.00

12. Project Summary — Skills Progression

This case study covers the full spectrum of SQL skills required for a professional data analyst role, progressing from basic syntax to strategic business intelligence.

Level	Skills Demonstrated	Queries
Level 1 — Basic Exploration	WHERE, ORDER BY, LIMIT, Filtering	Q1, Q2, Q3
Level 2 — Aggregation	COUNT, SUM, GROUP BY, HAVING	Q4, Q5, Q6
Level 3 — Joins	INNER JOIN, multi-table JOIN patterns	Q7, Q8, Q9
Level 4 — Profit Analytics	Arithmetic expressions, 4-table JOINs	Q10, Q11
Level 5 — Customer Intelligence	Subqueries, nested SELECT, HAVING	Q12, Q13
Level 6 — Strategic BI	MONTH(), date functions, revenue ranking	Q14, Q15, Q16
Capstone — Full Simulation	All of the above combined in P&L reports	5 Reports

--- End of Assignment ---

Submitted by: Chetan Prajapat

ACE_Project_ChetanPrajapat